



Δομές Δεδομένων
Διδάσκουσα: Κ. Παπακωνσταντινοπούλου
Χειμερινό Εξάμηνο 2024

Εργασία 1

Ουρές: Υλοποιήσεις ΑΤΔ και εφαρμογές
Προθεσμία υποβολής: Τρίτη 17/12/2024, 23:59

Σκοπός της εργασίας αυτής είναι η εξοικείωση με βασικούς αφηρημένους τύπους δεδομένων όπως οι στοιβές και οι ουρές FIFO. Η εργασία αποτελείται από δύο υλοποιήσεις ΑΤΔ (Μέρος Α), δύο εφαρμογές (Μέρος Β και Γ) και μία αναφορά (Μέρος Δ). Διαβάστε προσεκτικά την εκφώνηση και τα ζητούμενα της εργασίας.

Μέρος Α [30 μονάδες]. Στον φάκελο «Εγγραφα/Εργασίες/Εργασία 1» του eclass, δίνονται οι διεπαφές `StringStack` και `DoubleQueue`, που δηλώνουν τις βασικές μεθόδους για μια στοιβα και μια ουρά FIFO, καθεμία με στοιχεία του τύπου που υπάρχει ως πρόθεμα στο όνομά της. Δημιουργήστε μία υλοποίηση των ΑΤΔ `StringStack` και `DoubleQueue`, δηλαδή γράψτε δύο κλάσεις που υλοποιούν τις δύο διεπαφές.

Οδηγίες υλοποίησης:

- Οι κλάσεις σας **πρέπει να ονομάζονται** `StringStackImpl` και `DoubleQueueImpl`.
- Η υλοποίηση και για τις δύο διεπαφές θα πρέπει να γίνει χρησιμοποιώντας λίστα μονής σύνδεσης (η στοιβα και η ουρά δηλαδή πρέπει να αποθηκεύουν το περιεχόμενό τους σε λίστα μονής σύνδεσης).
- Κάθε μέθοδος εισαγωγής ή εξαγωγής στοιχείου θα πρέπει να ολοκληρώνεται σε χρόνο $O(1)$, δηλαδή σε χρόνο ανεξάρτητο από το πλήθος των αντικειμένων που μπορεί να βρίσκονται μέσα στην ουρά. Ομοίως, η μέθοδος `size` θα πρέπει να εκτελείται σε $O(1)$.
- Όταν η στοιβα ή η ουρά είναι κενή, οι μέθοδοι που διαβάζουν από την δομή θα πρέπει να πετάνε εξαίρεση τύπου `NoSuchElementException`. Η εξαίρεση `NoSuchElementException` ανήκει στην core βιβλιοθήκη της Java. Κάντε την `import` από το πακέτο `java.util`. Μην κατασκευάσετε δική σας εξαίρεση.
- Μπορείτε να χρησιμοποιήσετε μέρος του κώδικα που έχει παρουσιαστεί στα εργαστήρια του μαθήματος (διαθέσιμος στο eclass) ή να γράψετε εξ' ολοκλήρου τη δική σας λίστα ή να ορίσετε μόνο αντικείμενα τύπου `Node` μέσα στην κλάση της ουράς, για τους κόμβους της λίστας, χωρίς να ορίσετε κλάση λίστας. Για να αποκτήσετε καλύτερη εξοικείωση

προτείνουμε να ξεκινήσετε από την αρχή και να γράψετε τις δικές σας κλάσεις (σίγουρα δεν θα χάσετε μονάδες όμως χρησιμοποιώντας ό,τι έχετε δει στο εργαστήριο).

- Δεν απαιτείται να υπάρχει μέθοδος `main` στα αρχεία που θα παραδώσετε για το Μέρος Α. Όμως, καλό είναι να φτιάξετε μια `main` όπου μπορείτε να τρέξετε μερικά παραδείγματα για να βεβαιωθείτε για την ορθότητα των μεθόδων σας (δεν θα αποτελεί μέρος της βαθμολόγησης ή `main` σας, καθώς θα χρησιμοποιήσουμε δικό μας πρόγραμμα για να αξιολογήσουμε τις υλοποιήσεις των διεπαφών).
- **Προαιρετικά:** μπορείτε να κάνετε την υλοποίησή σας με χρήση `generics` για να μπορείτε να χειρίζεστε ουρές και στοιβάδες που περιέχουν οποιονδήποτε τύπο αντικειμένων. Υπάρχει ένα 10% bonus σε όσους χρησιμοποιήσουν `generics` για το Μέρος Α. Αν κάνετε χρήση `generics`, πρέπει να τροποποιήσετε κατάλληλα τα `interfaces` που σας δίνονται για να δουλεύουν για οποιονδήποτε τύπο δεδομένων.
- Δεν επιτρέπεται να χρησιμοποιήσετε έτοιμες υλοποιήσεις δομών τύπου λίστας, στοιβάς, και ουράς, από την βιβλιοθήκη της Java (π.χ. `Vector`, `ArrayList` κλπ).

Μέρος Β [30 μονάδες]. Χρησιμοποιώντας την κατάλληλη από τις δύο υλοποιήσεις του μέρους Α, γράψτε ένα πρόγραμμα-πελάτη το οποίο θα κάνει έλεγχο για το ταίριασμα των ετικετών (`tags`) σε ένα αρχείο HTML. Σε κάθε αρχείο HTML, οι ετικέτες οριοθετούν τμήματα κειμένου. Μια ετικέτα ανοίγματος έχει τη μορφή `<tag_name>` ενώ η αντίστοιχη ετικέτα κλεισίματος είναι η `</tag_name>`. Κάποιες απλές και συνηθισμένες ετικέτες είναι οι εξής:

- `body`, για να ξεκινήσει το κυρίως σώμα του εγγράφου
- `h1`, για την επικεφαλίδα
- `p`, για ορισμό παραγράφου
- `ol`, για αριθμημένη λίστα στοιχείων
- `b`, για έντονη γραμματοσειρά

Ιδανικά, κάθε HTML έγγραφο έχει *ταιριασμένες* ετικέτες, αν δηλαδή υπάρχει κάπου η ετικέτα `<h1>` για την οποία υπάρχει αντίστοιχη ετικέτα κλεισίματος, θα πρέπει αργότερα στο αρχείο να εμφανίζεται η ετικέτα `</h1>`, που σηματοδοτεί το τέλος της επικεφαλίδας. Το ίδιο πρέπει να συμβαίνει για όλες τις ετικέτες, με εξαίρεση αυτές που εξ ορισμού δεν έχουν ετικέτα κλεισίματος. Επίσης, αν υπάρχουν εμφωλευμένες ετικέτες μέσα σε ένα αρχείο, θα πρέπει να κλείνουν με τη σωστή σειρά: θα πρέπει να κλείσει πρώτα η τελευταία ετικέτα που άνοιξε προτού κλείσουν οι υπόλοιπες. Αν και αρκετοί browsers παρουσιάζουν ανοχή στην ύπαρξη κάποιων μη ταιριασμένων ετικετών, θα θέλαμε να μην έχουμε τέτοιες ετικέτες στα αρχεία που επεξεργαζόμαστε.

Το πρόγραμμα-πελάτη που θα γράψετε θα πρέπει να διαβάζει γραμμή προς γραμμή ένα HTML αρχείο, το όνομα του οποίου θα δίνεται από τον χρήστη, και να αποφασίζει αν οι ετικέτες στο αρχείο είναι σωστά ταιριασμένες ή όχι, τυπώνοντας αντίστοιχο μήνυμα. Θεωρήστε ότι οι ετικέτες που εξ ορισμού δεν έχουν αντίστοιχη ετικέτα κλεισίματος είναι οι: `<br>` και `<img>`, και, προφανώς, όπου και να τις συναντήσουμε αυτές, τις θεωρούμε σωστά ταιριασμένες.

Οδηγίες υλοποίησης:

- Το πρόγραμμά σας **πρέπει να ονομάζεται** `TagChecking.java`.
- Θα πρέπει να χρησιμοποιήσετε την υλοποίηση της κατάλληλης δομής δεδομένων από το Μέρος Α.
- Για το διάβασμα του `html` αρχείου, μπορείτε να χρησιμοποιήσετε έτοιμες κλάσεις της Java, όπως οι `Scanner`, `FileReader`, `BufferedReader`, κλπ. Μπορείτε επίσης να χρησιμοποιήσετε έτοιμες μεθόδους για να επεξεργαστείτε μεταβλητές τύπου `String`. Πέρα από το διάβασμα όμως του αρχείου, το υπόλοιπο πρόγραμμα θα πρέπει να κάνει

χρήση της δομής δεδομένων που επιλέξατε παραπάνω για να αποφασίσει για το ταίριασμα των ετικετών.

- Η εργασία σας θα αξιολογηθεί πάνω σε html αρχεία με απλές ετικέτες ανοίγματος και κλεισίματος, σαν αυτές που αναφέρθηκαν παραπάνω. Π.χ. μην ανησυχείτε για tags που περιέχουν περισσότερη πληροφορία, όπως το: `<a href="http://...">link </a>`. Επίσης δεν χρειάζεται να ελέγχετε αν το html αρχείο έχει άλλα συντακτικά λάθη. Ασχοληθείτε μόνο με το ταίριασμα των ετικετών.
- Το πρόγραμμά σας θα έχει μέθοδο `main` και το μονοπάτι για το προς εξέταση html αρχείο θα δίνεται ως παράμετρος γραμμής εντολών κατά την εκτέλεση του προγράμματός σας (χρήση `args[0]` της `main`). Η εκτέλεση του προγράμματος θα πρέπει να είναι δυνατή είτε από τη γραμμή εντολών με μια εντολή της μορφής:

```
> java TagChecking path_to_html_file.html
```

είτε από το IDE σας (δείτε το αρχείο `Eclipse-command line arguments.pdf` με οδηγίες, στον φάκελο της Εργασίας 1).

Μέρος Γ [30 μονάδες]. Χρησιμοποιώντας την κατάλληλη από τις δύο υλοποιήσεις του μέρους Α, γράψτε ένα πρόγραμμα-πελάτη, το οποίο λύνει το εξής λογιστικό πρόβλημα: όταν θέλουμε να υπολογίσουμε το καθαρό κέρδος που προκύπτει από πώληση μετοχών, υπάρχει αμφισημία ως προς το ποιες μετοχές θεωρούμε ότι πουλάμε (όταν έχουμε μετοχές που έχουν αγοραστεί σε διαφορετικές τιμές). Για παράδειγμα, έστω ότι την χρονική στιγμή t_1 αγοράσαμε 40 μετοχές από κάποια εταιρεία, σε τιμή 25.2 ευρώ. Μετέπειτα, τη χρονική στιγμή t_2 , αγοράσαμε άλλες 30 μετοχές από την ίδια εταιρεία σε τιμή 21.8 ευρώ, και τη χρονική στιγμή t_3 , αγοράσαμε ακόμα 50 μετοχές σε τιμή 34.4 ευρώ. Έστω ότι αργότερα πουλάμε 100 μετοχές (από τις 120 που έχουμε) σε τιμή 30 ευρώ. Η λογιστική αρχή που χρησιμοποιείται για να καθορίσουμε το καθαρό κέρδος της πώλησης εκείνη τη στιγμή στηρίζεται στην ιδέα ότι οι μετοχές που πουλάμε θεωρούμε ότι είναι αυτές που αγοράστηκαν πιο νωρίς. Τα περισσότερα λογισμικά διαχείρισης επενδυτικών χαρτοφυλακίων χρησιμοποιούν αυτή την αρχή. Στο παράδειγμά μας, αυτό σημαίνει ότι από τις 100 μετοχές που πουλάμε, θεωρούμε ότι οι πρώτες 40 προέρχονται από αυτές που αγοράστηκαν πιο νωρίς, δηλαδή την στιγμή t_1 , οι επόμενες 30 από αυτές που αγοράστηκαν την στιγμή t_2 , και οι επόμενες 30 από την τελευταία αγορά. Έτσι το καθαρό κέρδος μας εκείνη τη στιγμή θεωρούμε ότι είναι:

$$40(30 - 25.2) + 30(30 - 21.8) + 30(30 - 34.4) = 192 + 246 - 132 = 306$$

Σημειώστε ότι στον παραπάνω υπολογισμό, δε λαμβάνουμε υπόψη στο κέρδος τις 20 μετοχές που έχουν περισσέψει από τις 120, μιας και δεν τις έχουμε πουλήσει προς το παρόν.

Γράψτε ένα πρόγραμμα το οποίο θα διαβάζει από ένα `.txt` αρχείο μια ακολουθία από διαδοχικές συναλλαγές της μορφής:

```
buy 40 price 25.2
buy 30 price 21.8
buy 50 price 34.4
sell 100 price 30
buy 25 price 35.6
buy 15 price 32.7
sell 30 price 40.1
...
```

και θα τυπώνει το συνολικό καθαρό κέρδος ή ζημιά που προκύπτει ακριβώς μετά από την τελευταία πώληση. Το παραπάνω αρχείο απεικονίζει τις συναλλαγές με τη σειρά που γίνονται,

δηλαδή στο παράδειγμα αυτό έγιναν πρώτα 3 αγορές, μετά μία πώληση, μετά 2 αγορές και μετά ξανά μία πώληση. Το αρχείο θα ξεκινά πάντα με τουλάχιστον μια συναλλαγή αγοράς και θα τελειώνει πάντα με συναλλαγή πώλησης.

Οδηγίες υλοποίησης:

- Το πρόγραμμά σας **πρέπει να λέγεται** `NetProfit.java`.
- Για το διάβασμα του αρχείου εισόδου, μπορείτε να χρησιμοποιήσετε μεθόδους αντίστοιχες με αυτές που αναφέρθηκαν στο Μέρος Β. Στη συνέχεια θα πρέπει να χρησιμοποιήσετε την υλοποίηση κατάλληλης δομής δεδομένων του Μέρους Α, για να υπολογίσετε το καθαρό κέρδος.
- Θεωρούμε ότι σε όλες τις γραμμές του αρχείου εισόδου, το πλήθος των μετοχών είναι ακέραιοι αριθμοί και οι τιμές είναι πραγματικοί αριθμοί. Δεν απαιτείται να ελέγξετε ότι όλα τα στοιχεία του αρχείου εισόδου είναι σε σωστή μορφή (οι εργασίες θα αξιολογηθούν με είσοδο αρχεία με τη μορφή του παραπάνω παραδείγματος). Θα πρέπει όμως να τυπώνετε μήνυμα λάθους αν μέσα στο αρχείο υπάρχει πώληση πλήθους μετοχών το οποίο δεν είναι διαθέσιμο.
- Το μονοπάτι προς το αρχείο εισόδου θα δίνεται ως όρισμα, σε αντιστοιχία με το Μέρος Β. Π.χ. η εκτέλεση από τη γραμμή εντολών θα γίνεται ως εξής:

```
> java NetProfit path_to_text_file.txt
```

Μέρος Δ - Αναφορά παράδοσης [10 μονάδες]. Ετοιμάστε μία σύντομη αναφορά σε pdf αρχείο (μην παραδώσετε Word ή txt αρχεία!) με όνομα `project1-report.pdf`, στην οποία θα αναφερθείτε στα εξής:

- a. Εξηγήστε συνοπτικά πώς υλοποιήσατε τις διεπαφές στο μέρος Α (άνω όριο 2 σελίδες).
- b. Για το μέρος Β, εξηγήστε πώς χρησιμοποιήσατε την υλοποίηση από το μέρος Α για να φτιάξετε το πρόγραμμα που ζητείται (άνω όριο 2 σελίδες).
- c. Ομοίως για το μέρος Γ (άνω όριο 2 σελίδες).
- d. Παραθέστε οποιαδήποτε πληροφορία κρίνετε απαραίτητη για να γνωρίζουν οι βοηθοί του μαθήματος πώς εκτελείται ο κώδικάς σας (π.χ. επιβεβαιώστε ότι χρησιμοποιήσατε τις οδηγίες για τα command line arguments, για να δίνετε σαν είσοδο το όνομα του αρχείου στο Μέρος Β).

Το συνολικό μέγεθος της αναφοράς θα πρέπει να είναι τουλάχιστον 2 σελίδες και το πολύ 6 σελίδες.

Οδηγίες Παράδοσης

Η εργασία σας θα πρέπει να μην έχει συντακτικά λάθη και να μπορεί να μεταγλωττίζεται. **Εργασίες που δεν μεταγλωττίζονται χάνουν το 50% της συνολικής αξίας. Η διόρθωση των εργασιών από τους βοηθούς του μαθήματος αφορά την αξιολόγηση μεταγλωττίσιμων υλοποιήσεων και όχι την εύρεση λαθών μεταγλώττισης.**

Η εργασία θα αποτελείται από:

1. Τον πηγαίο κώδικα (source code). Τοποθετήστε σε ένα φάκελο με όνομα **src** τα αρχεία `java` που έχετε φτιάξει. Χρησιμοποιήστε τα ονόματα των κλάσεων όπως ακριβώς δίνονται στην εκφώνηση. Επιπλέον, φροντίστε να συμπεριλάβετε όποια άλλα αρχεία πηγαίου κώδικα απαιτούνται για να μεταγλωττίζεται η εργασία σας. Φροντίστε επίσης να προσθέσετε επεξηγηματικά σχόλια όπου κρίνετε απαραίτητο στον κώδικά σας.
2. Την αναφορά παράδοσης.

Όλα τα παραπάνω αρχεία θα πρέπει να μπουν σε ένα αρχείο zip. Το όνομα που θα δώσετε στο αρχείο αυτό θα είναι ο αριθμός μητρώου σας πχ. 3210056_3210066.zip. Στη συνέχεια, θα υποβάλετε το zip αρχείο σας στην περιοχή του μαθήματος «Εργασίες» στο e-class. Δεν χρειάζεται υποβολή και από τους 2 φοιτητές μιας ομάδας. Φροντίστε να υπάρχουν οι αριθμοί μητρώου και των δύο μελών της ομάδας στο όνομα του zip αρχείου, καθώς επίσης να υπάρχουν τα ονοματεπώνυμα και οι αριθμοί μητρώου και των δύο μελών της ομάδας στην αναφορά. Αν δεν υπάρχει το όνομά σας, δε μπορείτε να διεκδικήσετε βαθμό στην εργασία αυτή.